Technische Hochschule Ingolstadt

Faculty of Electrical and Information Technology

Master degree in International Automotive Engineering

Master Thesis

Design and configuration of Freedom from interference concept using Memory Protection Unit

in collaboration with:
Conti Temic Microelectronics GmbH

Author:
Younis Ebaid

First Examiner:
Arnold Armin

Submitted on: September 09, 2024

Declaration

I declare that I have authored this thesis independently, that I have not used other than the declared sources/resources, that I have not presented it elsewhere for examination purposes, and that I have explicitly indicated all material which has been quoted either literally or by consent from the sources used. I have marked verbatim and indirect quotations as such.

Ingolstadt, _____

Younis Ebaid

Acknowledgments

I would like to thank everyone who supported me throughout this long journey of acquiring my Master's degree. It has been the toughest journey I have taken so far. Special thanks go to my supervisor, Arnold Armin, for always being patient and understanding. To my mother, my sister, and my late father, who mean the world to me

Younis Ebaid
Ingolstadt, Germany
September, 2024

Abstract

The automotive industry is constantly growing, adding more features to new models to meet customer demands for increased comfort, safety, and security.
In this thesis, we focus on the safety aspect by examining the ISO 26262 safety standards, their components, applications, and the various methods available in the industry to ensure freedom from interference between the different software components in the vehicle's ECU.

We then delve deeper into the concept of the mixed-criticality approach and the required hardware components needed to implement this concept.
The MPU is a critical hardware component that safeguards against unintended or unauthorized access to memory, thereby preventing potential system failures and ensuring the integrity of data. This research explores the design principles of an MPU, emphasizing its role in mitigating risks identified in the ISO 26262 safety lifecycle. A detailed examination of the MPU's architecture is provided, including the segmentation of memory regions, the configuration of access permissions, and the enforcement of protection rules at runtime.

The results indicate that the implemented MPU enhances the safety of the embedded system by effectively preventing unsafe memory operations, thus contributing to the overall Safety Integrity Level (SIL) achievement as per ISO 26262 standards. This work not only provides a comprehensive framework for MPU design in safety-critical applications but also serves as a reference for future developments in the field of embedded system safety.

Acronyms

ABS Anti-lock Braking System.

ADAS Advanced Driver-Assistance Systems.

ASIL Automotive Safety Integrity Levels.

AUTOSAR Automotive Open System Architecture.

BSW Basic Software.

C Controllability.

CAN Controller Area Network.

CPU Central Processing Unit.

E Exposure.

ECU Electronic Control Unit.

FFI Freedom From Interference.

FSC Functional Safety Concept.

HARA Hazard Analysis and Risk Assessment.

HW Hardware.

I/O Input/Output.

IDE Integrated Development Environment.

IEC International Electrotechnical Commission.

ISR Interrupt Service Routine.

LIN Local Interconnect Network.

MCAL Microcontroller Abstraction Layer.

MMU Memory Management Unit.

MPU Memory Protection Unit.

NMI Non-Maskable Interrupt.

OEM Original Equipment Manufacturer.

OS Operating System.

PC Program Counter.

PES Programmable Electronic Safety.

PTE Page Table Entry.

RAM Random Access Memory.

RTE Runtime Environment.

RTOS Real Time Operating System.

S Severity.

SIL Safety Integrity Levels.

SMT Software Managed Task.

SPI Serial Peripheral Interface.

SWC Software Component.

TAE Temporal Asynchronous Exception.

TIM Trap Identification Number.

TSC Technical Safety Concept.

UART Universal Asynchronous Receiver-Transmitter.

WdgDrv Watchdog Driver.

WdgIF Watchdog Interface.

WdgM Watchdog Manager.

WHO World Health Organization.

List of figures

List of tables

Table of contents

1 Introduction

Development in the automotive industry has slowly changed from developing a purely mechanical product at it is beginning into a product where most innovations are related to E/E system coupled with software or what is known as embedded systems.
This change was fueled partly by the limitations of some of the mechanical solutions compared to their E/E counterparts and partly by the desire to innovate new products to meet customer expectations in a digitized world and meet environmental and regulatory standards.
This trend is expected to continue in the near future at a breakneck pace to keep up with the 4 biggest trends in the current automotive world:

1-Autonomous driving
2-Electrification
3-connectivity
4-shared mobility.

Although modern road vehicles have come a long way in terms of development and the features they can offer compared to their older siblings, a problem still exists which is deaths and injure related to roads.
According to a World Health Organization (WHO) survey in June 2022, there is an estimated 1.3 million death on roads yearly and between 20 and 50 million more people suffer non-fatal injuries with many incurring a disability as a result of their injury.

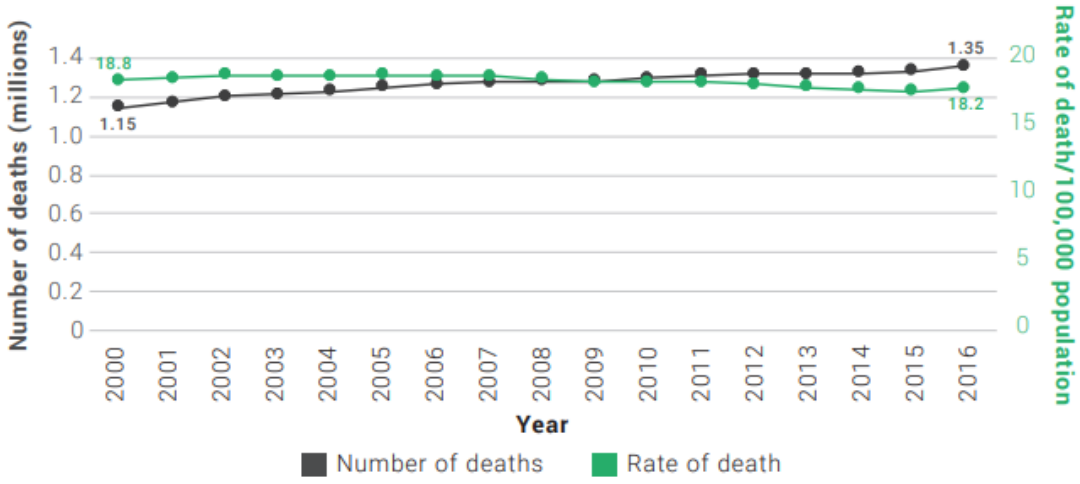

Figure 1: WHO Death survey results [1]

The desire to prevent and reduce these fatalities and injuries has led to an increase in safety-critical systems within cars. Safety-critical systems are defined as systems where

the interaction between cyber-physical components and the physical world, if malfunctioning, could lead to injury, loss of life, or property damage. can be tackled using different methods often combined together which are: regulation, E/E safety systems such as the ABS, ESC... etc, or schemes that helps in identifying malfunctions (electric and electronic), and specify actions and techniques to be adopted to mitigate risks and damage during instances of software or hardware failures.
The last method is referred to as Functional safety. In summary, there are three main subgoals for functional safety:
1. Preventing systematic failures
2. Mitigating random failures
3. Showing (providing evidence) that the previous goals have been achieved.

Functional Safety aims to ensure that technological developments operate under a safety umbrella. It acts as a shield, protecting users from modern technology and safeguarding technology from improper handling by users. This is achieved by introducing processes and regulations that must be followed by all stakeholders to ensure the safe operation of vehicles and their subsystems under normal conditions, as well as their ability to manage various types of failures or human errors.

The development of an automotive system is distributed among many organizations within the automotive industry's value chain. This distributed development adds another layer of complexity, as the responsibilities for achieving functional safety must also be shared across the value chain. The diversity of stakeholders, combined with the increasing number of E/E systems, has led to the need for a unified safety standardization. The goal of this standardization is to minimize the risks associated with product design and development by providing recommended requirements and best practices.
Currently, ISO 26262 is the unified safety standard that the automotive industry follows. Achieving functional safety is a critical factor for the social acceptance of automated driving and smart mobility technologies.

1.1 Problem statement

In the realm of embedded systems, particularly those deployed in safety-critical applications such as automotive systems, ensuring the reliability and safety of operations is paramount. As these systems grow in complexity, the potential for software errors, unintended memory access, and unauthorized manipulation of critical data also increases. Such vulnerabilities can lead to system failures with severe consequences, including loss of life, substantial financial loss, and damage to property.
Implementing safety systems in compliance with ISO 26262 presents several challenges. One significant challenge is that safety-related software often needs to coexist with non-safety software, leading to a coexistence problem. This issue can be addressed either by developing the entire system in accordance with the highest required safety standards, known as Maximum ASIL Design, or by implementing a mixed system, referred to as a

mixed-criticality system.

The co-existence of a safety critical and non-safety critical software in a single ECU is an unavoidable consequence of the complexity of todays modern automotive systems, this coexistence if not addressed properly might lead to sever safety hazards. One solution to this problem which this thesis aims at is to add a mechanism that organizes this coexistence to prevent any unwanted interference.

1.2 Objective

Memory protection units are playing increasing roles in safety related applications however, as resources in embedded systems are limited, new approaches are needed to maximize the safety and security applications while reducing the needed CPU resources needed. This thesis focuses on designing the rules for how the Memory Protection Unit (MPU) will handle different code access authorizations based on ASIL levels, and implementing this configuration on an AURIX microcontroller to demonstrate a proof of concept.

1.3 Outline

Chapter 2 provides the necessary theoretical background needed to follow the rest of the thesis.

Chapter 3 takes a closer look at ISO 26262, the primary functional safety standard for the automotive industry, which is the focus of this work.

Chapter 4 describes the different approaches for solving the problem described and provides a comparison for the described approaches.

Chapter 5 outlines the selection of an approach and the implementation of the configuration on the chosen hardware.

Chapter 6 presents the final results

2 Literature review

2.1 Introudction

The literature on functional safety and the Memory Protection Unit (MPU) in the automotive domain is extensive and reflects the growing complexity and criticality of modern vehicle systems. As automotive technology evolves, vehicles are increasingly reliant on electronic control units (ECUs) to perform both essential and advanced functions, such as braking, steering, and autonomous driving. Ensuring the safety and reliability of these functions is paramount, especially when considering the stringent safety standards like ISO 26262, which governs the functional safety of electrical and electronic systems in road vehicles.

2.2 Literature review

Arena et al. (2023) conducted a case study focusing on improving spatial isolation in mixed-ASIL architectures within the AUTOSAR OS to ensure freedom from interference. The study identified that simple write operations by low ASIL cores on shared OS data structures could adversely impact higher ASIL cores, potentially leading to system malfunctions. To mitigate these risks, the authors implemented several safety mechanisms, including redundancy checks and reformatting internal data structures to minimize unnecessary sharing among cores. Additionally, they introduced a Remote Procedure Call (RPC) mechanism to ensure that critical operations initiated by lower ASIL cores are executed by higher ASIL cores, thus maintaining spatial isolation. Performance evaluations on a Tricore architecture indicated that while redundancy mechanisms increased overhead, the RPC approach introduced only an acceptable overhead, effectively balancing safety and performance in mixed-ASIL environments.[2]

Su et al. (2019) proposed a novel approach to ensuring Freedom from Interference (FFI) in Mixed-Criticality Systems (MCS) by utilizing causal learning techniques to analyze in-field anomaly data. Traditional FFI analysis methods, which primarily focus on the product development phase and are largely qualitative, have struggled with comprehensive coverage of potential interference, particularly in complex environments and under varying usage conditions.
These methods also face difficulties in distinguishing cascading failures from common-source failures. To address these challenges, the authors introduced a data-driven method that mines real-life anomaly data to identify potential interference issues that might be missed by conventional approaches. Their method offers a more robust verification of FFI by quantitatively measuring causal relationships between system elements with different criticality levels, thus improving the reliability and safety of MCS, particularly in safety-critical domains such as automotive and aerospace applications.
The approach demonstrated effectiveness in identifying interference in scenarios that existing solutions found challenging to analyze, offering a significant advancement in FFI

verification methods.[3]

Ficek et al. (2012) explored the application of the AUTOSAR Timing Protection mechanism to enhance the safety and efficiency of ISO 26262-compliant mixed-criticality systems.
The authors evaluated three priority-assignment strategies: Rate-Monotonic Scheduling (RMS), Criticality-As-Priority-Assignment (CAPA), and a hybrid approach combining elements of both. They found that while RMS is resource-efficient, it does not prevent criticality inversion, potentially compromising higher criticality tasks. On the other hand, CAPA ensures no criticality inversion but at the cost of resource efficiency. By leveraging the AUTOSAR Timing Protection service, which monitors execution times and other parameters, the hybrid approach was shown to effectively balance safety and resource efficiency.
The method proposed for configuring the Timing Protection service, particularly in setting Timing Protection Budgets (TPB), was validated through a series of experiments, demonstrating that careful configuration can mitigate error propagation while maintaining system performance.[4]

2.3 Conclusion

This literature review highlights key works in the field of Automotive Functional Safety with a focus on freedom from interefence applications, . The references cover a wide range of topics, including Functional safety, AUTOSAR based safety applications, Safety protocools and standards. These works provide a solid foundation for understanding the challenges and opportunities in Mixed criticality systems within the complex automotive ecosystem. The next steps involve leveraging this knowledge to develop a robust concept for Memory protection Unit within a Hardware-in-the-Loop environment, ensuring the reliability, safety of data within the system.

3 Automotive Functional Safety

3.1 Background

The first safety standardization effort began when the International Electrotechnical Commission (IEC) set up a task group in 1985 to assess the viability of developing a generic standard for programmable electronic systems to be used in safety applications. The outcome of this assessment was the establishment of a working group to develop a holistic, systems-based approach. This process culminated in 1998 with the introduction of IEC 61508, which is widely regarded as the father of safety standardization.
IEC 61508 is an international standard for managing Functional Safety with regard to Electrical / Electronic / Programmable Electronic Safety Related Systems (referred to as E/E/PES). The standard has been in development for a number of years and the last of the seven parts were published in 2000. The standard is generic in nature and can be applied to a safety related application in any industry sector.
The standard defines a rational and consistent approach to achieving functional safety and uses the concept of the 'safety lifecycle' (from concept design, through hazard and risk analysis, specification, implementation, operation, maintenance to eventual decommissioning) as a framework for addressing the phases to achieve functional safety in a systematic and auditable manner.
The standard helps determine Safety Integrity Levels (SIL). There are four SILs:
SIL1, SIL2, SIL3 and SIL4 as shown in the following table [5]

Safety Integrity Level	Probability failure on demand	Risk Reduction Factor
SIL 4	$\geq 10^{-5}$ to $< 10^{-4}$	100000 to 10000
SIL 3	$\geq 10^{-4}$ to $< 10^{-3}$	10000 to 1000
SIL 2	$\geq 10^{-3}$ to $< 10^{-2}$	1000 to 100
SIL 1	$\geq 10^{-2}$ to $< 10^{-1}$	100 to 10

Table 1: Safety Integrity Levels.

The risk of failure increases with each respective SIL. Conducting a risk assessment determines which SIL is required. A SIL is determined by examining the Systematic Capability, Architectural Constraints, and the Probability of Dangerous Failure.
Despite the success of IEC 61508 it had two limitations which prevented it from being used in automotive industry
1-It is generic
2-Assumes that everything is developed by a single entity which is not the case in automotive systems as various stakeholders are involved from OEMs, to tier 1 suppliers to various services companies.
The standard has two fundamental principles:

An engineering process called the safety life cycle is defined based on best practices in order to discover and eliminate design errors and omissions. A probabilistic failure approach to account for the safety impact of device failures.

The safety life cycle has 16 phases which roughly can be divided into three groups as follows:

- Phases 1–5 address analysis.
- Phases 6–13 address realisation.
- Phases 14–16 address operation.

The standard has seven parts:

- Parts 1–3 contain the requirements of the standard (normative).
- Part 4 contains definitions.
- Parts 5–7 are guidelines and examples for development and thus informative.

The IEC 61508 standard states: "To conform to this standard, it shall be demonstrated that the requirements have been satisfied to the specified criteria and that, for each clause or sub-clause, all the objectives have been met.
During actual application of this standard, the proof of compliance often demands listing all the IEC 61508 requirements along with an explanation of how each of them has been achieved. This sticks to both, products developed to meet IEC 61508 and specific application projects that would want to adhere themselves according to this standard.
As IEC 61508 is just a standard and not a law, adhering to its compliance is not always accompanied with a legal document. However, compliance in best practices is cited in most liability cases, for e.g. in contracts.
The global acceptance of this standard led to many countries hard-wiring this standard directly into their safety codes.
Moving forward with the goal of safety, there are numerous industry and government contracts for safety equipment, systems, and services that specifically require compliance with the IEC 61508 standard.

The IEC 61508 standard recommends the use of a functional safety lifecycle. While any safety lifecycle can be used, the standard provides a nominal 16-step process, which can be divided into three main classifications as an example.
The Analysis phase of the lifecycle deals with gathering background information to identify and specify the needs for the system.
The Realisation phase concerns system design and fabrication, while the Operation phase deals with using and maintaining the system properly during its operating life.[6]

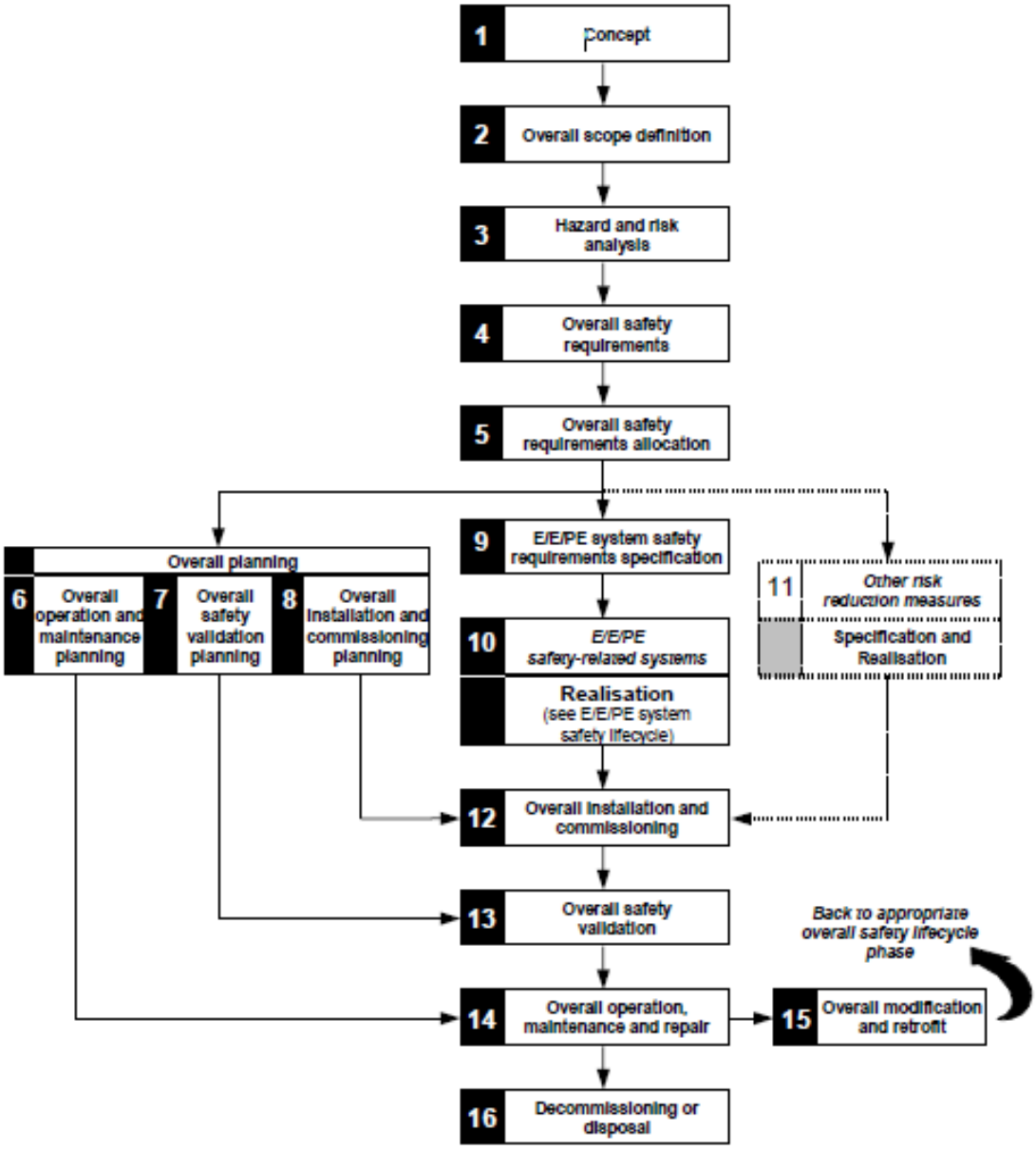

Figure 2: Overall safety lifecycle for IEC-61508.

3.1.1 Automotive Functional Safety standard ISO 26262

ISO 26262, titled "Road vehicles – Functional safety", is an international standard for functional safety of electrical and/or electronic systems that are installed in serial production road vehicles (excluding mopeds), defined by the International Organization for Standardization (ISO) in 2011, and revised in 2018.

ISO 26262 offers adaptation of IEC 61508's SIL levels into ASIL level Severity: estimate of the extent of harm to one or more individuals that can occur in a potentially hazardous event.

Provides an automotive safety lifecycle through the stages of management, development, production, operation, service until decommissioning. It also supports tailoring these steps according to the requirements during the lifecycle
Provides an automotive specific risk-based idea to classify risks This is termed under Automotive Safety Integrity Levels The ASIL levels are used for further specifying necessary safety requirements of a product so as to attain the acceptable residual risk It also provides requirements for validation, verification and confirmation measures to assure an acceptable and sufficient amount of safety is being reached

- Exposure (E): This is the measure of the possibilities of the vehicle being in a hazardous or risky situation that can cause harm to people and property. Various levels of exposure such as E1: very low probability, E2: low probability, E3: medium probability, E4: high probability are assigned to the automotive component being evaluated.
- Controllability (C) : Determines the extent to which the driver of the vehicle can control the vehicle if a safety goal is breached due to failure or malfunctioning of any automotive component being evaluated. The order of controllability is defined as: C1<C2<C3 (C1 for easy to control while C3 for difficult to control).
- Severity (S): Defines the seriousness or intensity of the damage or consequences to the life of people (passengers and road users) and property due to safety goal infringement. The order of severity is : S1 for light and moderate injuries; S2 for severe and life-threatening injuries, and S3 for life-threatening incidences.

Severity	Exposure	Controllability		
		C1 - Simple	C2 - Normal	C3 - Difficult
S1 - Light	E1 (very low)	QM	QM	QM
	E2 (low)	QM	QM	QM
	E3 (medium)	QM	QM	A
	E4 (high)	QM	A	B
S2 - Severe	E1 (very low)	QM	QM	QM
	E2 (low)	QM	QM	A
	E3 (medium)	QM	A	B
	E4 (high)	A	B	C
S3 - Fatal	E1 (very low)	QM	QM	A
	E2 (low)	QM	A	B
	E3 (medium)	A	B	C
	E4 (high)	B	C	D

Figure 3: ASIL levels.

The standard is divided into several parts, each addressing specific aspects of automotive functional safety.
Part 1 defines the vocabulary used throughout the standard, ensuring consistent terminology.

Part 2 focuses on the management of functional safety, outlining the organizational responsibilities and processes necessary for ensuring safety across all phases of a vehicle's lifecycle.

Part 3 addresses the concept phase, including hazard analysis and risk assessment, which are crucial for determining the safety goals and Automotive Safety Integrity Levels (ASILs).

Part 4 covers product development at the system level, detailing the requirements for designing and validating safe systems.

Part 5 and Part 6 are dedicated to hardware and software development, respectively, specifying guidelines for ensuring safety in electronic hardware and software components.

Part 7 involves production, operation, service, and decommissioning, ensuring safety is maintained even after the vehicle is in use.

Part 8 provides supporting processes such as configuration management, change management, and documentation, which support the implementation of functional safety

Finally, Part 9 focuses on the safety analysis techniques required to validate the safety measures, and Part 10 offers guidelines and recommendations for applying the ISO 26262 standard across various contexts in automotive development. Together, these parts create a comprehensive framework for managing and mitigating risks associated with electronic and electrical systems in vehicles, ensuring the highest levels of safety for road users.

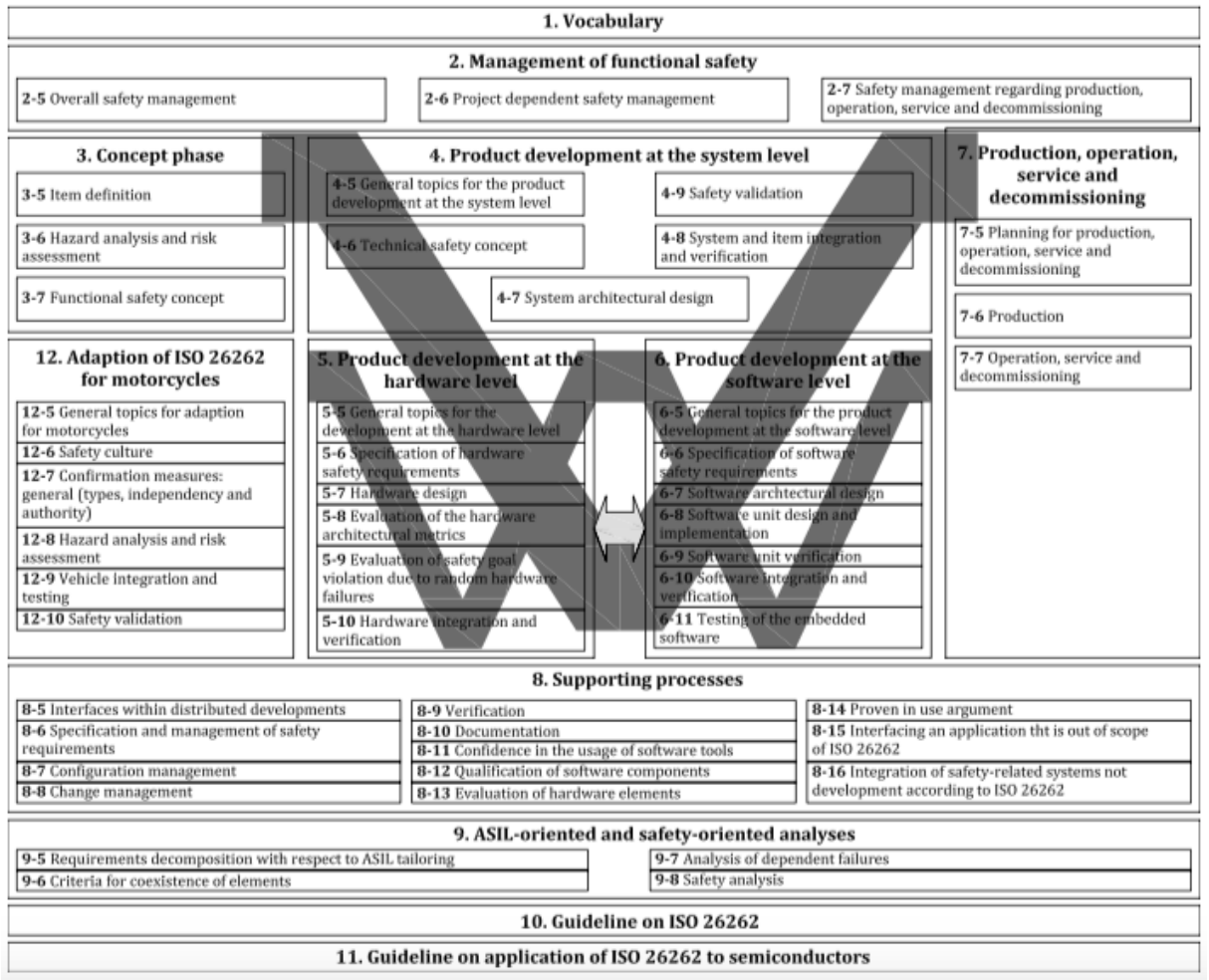

Figure 4: Components of ISO 26262

ISO 26262 is crucial in the automotive industry because it provides a comprehensive framework for ensuring the functional safety of electrical and electronic systems in vehicles, which are becoming increasingly complex with advancements in technology.

As modern vehicles rely more heavily on software and electronic control systems for critical functions such as braking, steering, and driver assistance, the potential for system failures that could lead to accidents has increased.
ISO 26262 addresses this by establishing standardized safety processes and practices to identify potential hazards, assess risks, and implement appropriate safety measures throughout the vehicle's development lifecycle.
This not only helps in reducing the likelihood of system failures but also ensures compliance with legal and regulatory requirements, thereby protecting manufacturers from liabilities.
Moreover, adhering to ISO 26262 fosters a culture of safety within organizations, promotes consistency in safety standards across the automotive industry, and enhances consumer confidence in vehicle safety, ultimately contributing to safer roads and saving lives. [7]

3.1.2 Safety lifecycle in ISO 26262

The Safety Lifecycle within ISO 26262 is a structured process integral to the development of embedded systems in the automotive industry. It starts with the identification of potential hazards during the concept phase, where risks associated with electrical and electronic (E/E) systems are assessed. This phase involves defining the Automotive Safety Integrity Levels (ASILs), which categorize the severity, exposure, and controllability of potential risks. Once hazards are identified and classified, the lifecycle proceeds to system design, where safety requirements are derived and incorporated into the system architecture. This ensures that safety mechanisms are built into the design, preventing or mitigating failures. The process continues through detailed design and implementation, where each component is designed to meet the established safety requirements, and then through rigorous verification and validation activities to confirm that the system behaves as expected under all operating conditions.

Throughout the Safety Lifecycle, ISO 26262 emphasizes traceability and documentation, ensuring that every decision and change is recorded, providing a clear audit trail. The lifecycle is iterative, meaning that safety assessments and validations are revisited at each stage of development, especially as changes are made or new information arises. This approach is critical in the context of embedded systems, where software and hardware integration is complex, and any failure could have severe consequences. By adhering to the ISO 26262 standard, automotive developers can ensure that their embedded systems not only meet functional requirements but also achieve the highest levels of safety, reducing the risk of system failures that could lead to accidents or harm.[7]

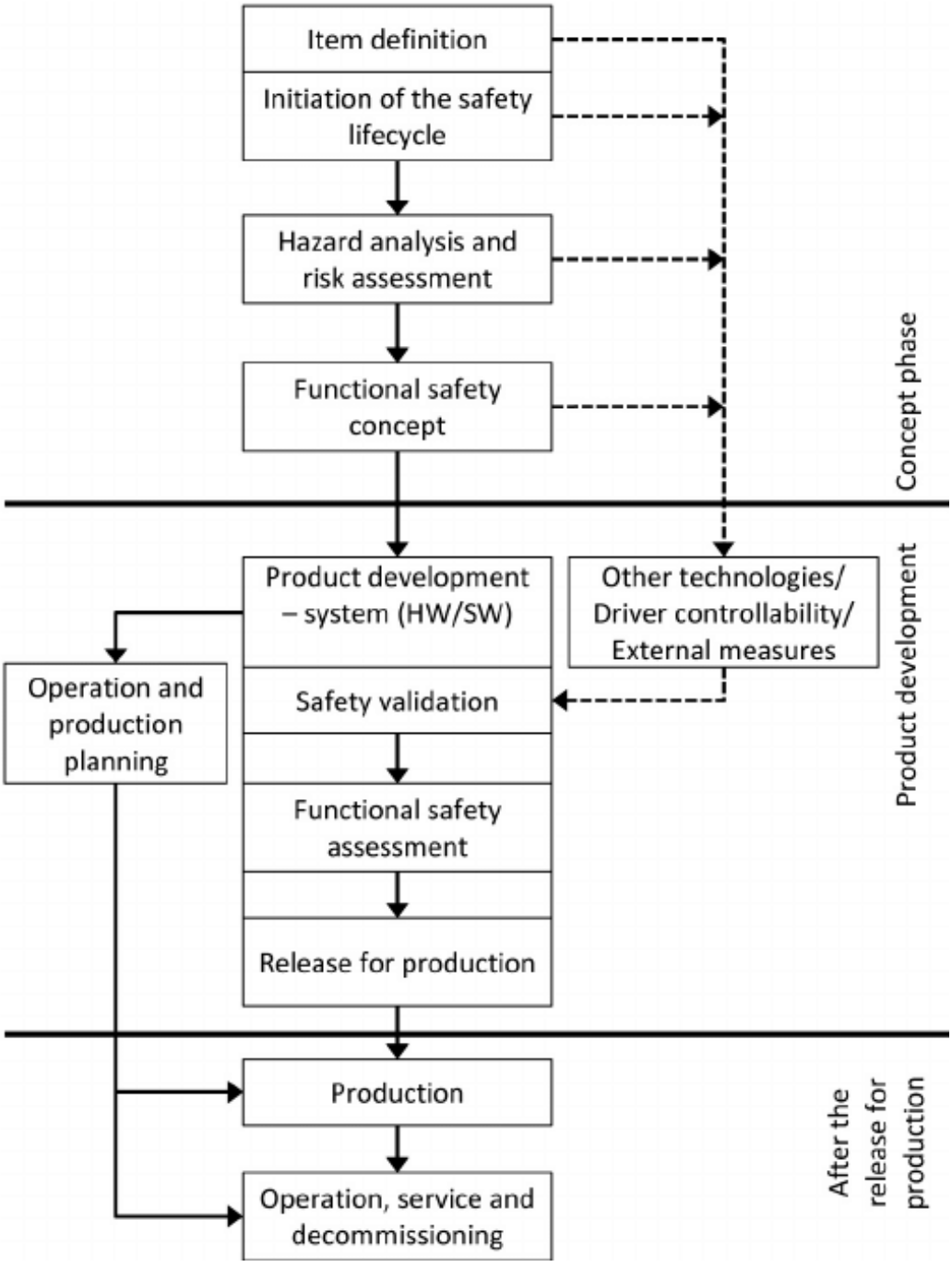

Figure 5: ISO 26262 Safety Lifecycle

3.1.3 ASIL Determination in ISO 26262

ASIL (Automotive Safety Integrity Level) determination within ISO 26262 is a critical process that helps classify the safety requirements of automotive systems based on the severity, exposure, and controllability of potential hazards. The ASIL is determined through a Hazard Analysis and Risk Assessment (HARA), which evaluates potential failures in the vehicle's electrical and electronic (E/E) systems. The severity considers the

potential impact of a failure on human life, the exposure assesses how frequently the hazardous situation might occur, and the controllability examines the driver's ability to prevent harm if the failure happens. Based on these factors, hazards are classified into four ASIL levels—A, B, C, and D—with ASIL D representing the highest risk, requiring the most stringent safety measures.

In the context of embedded systems development, ASIL determination guides the design and implementation of both hardware and software components. Higher ASIL levels impose stricter design constraints and require more rigorous verification and validation processes to ensure that the system can handle failures without compromising safety. For example, an ASIL D system might necessitate redundant components, fail-safe mechanisms, and comprehensive testing strategies to mitigate the risks of failure. The ASIL level directly influences the development lifecycle, as it dictates the safety measures that must be integrated into the system architecture, coding standards, and testing procedures, ensuring that the final product adheres to the required safety standards and reduces the likelihood of catastrophic failures.[7]

3.2 Coexistensoe of safety and non safety software within ISO 26262

The coexistence of safety-critical and non-safety software modules within a single automotive Electronic Control Unit (ECU) presents a significant challenge in the automotive industry.
Modern vehicles increasingly rely on complex ECUs that integrate various software components to control diverse functions, ranging from critical safety features like braking and airbag deployment to non-safety functions such as infotainment and navigation systems. This integration is driven by the need to reduce hardware costs and improve system efficiency.
However, combining safety and non-safety software within the same ECU raises serious concerns about functional safety, as the malfunction of a non-safety module could potentially interfere with the operation of safety-critical components, leading to hazardous situations.
Ensuring the reliable operation of safety functions while maintaining the desired functionality of non-safety software requires robust design methodologies, stringent partitioning, and strict adherence to safety standards such as ISO 26262.
This introduces complexities in software architecture, resource management, and verification and validation processes, necessitating advanced strategies to manage the interactions between diverse software modules to guarantee both safety and performance.[8]

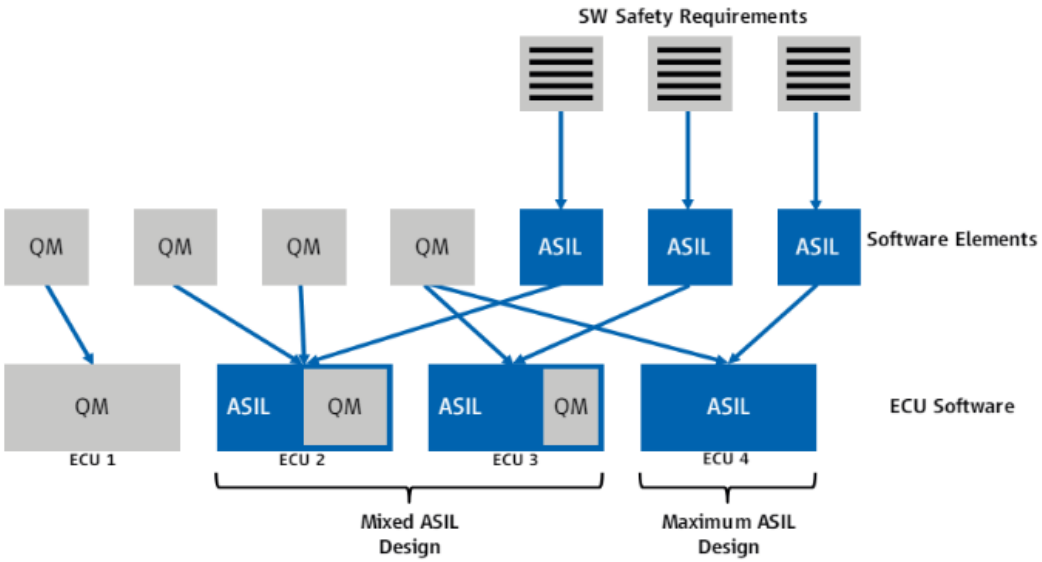

Figure 6: Mixed software problem.
[8]

3.2.1 Maximum ASIL Design

This approach is used when the non safety elements are small in quantity and it works by developing the whole software in accordance with the highest available ASIL level, it can also be feasible when the ASIL level is not high (e.g. ASIL B).

The "Maximum ASIL Design" has its advantages in use cases where a high share of the software provides safety-related functionality. In this approach, both the safety-related and the non-safety-related functions follow the development process of the highest ASIL in the system.
For the non-safety-related software elements, the coexistence argumentation follows a process argumentation: if those software elements are developed in the same stringent way applying the same process methods as the safety-related software elements, the coexistence of the elements is possible without further technical separation measures.
The only difference between the non-safety-related and the safety-related software elements is then the necessary safety analysis for the latter.
Compared to the "Mixed ASIL Design" this approach gives the following benefits:
• No additional complexity for the development of a partitioning concept.
• No performance penalty due to safety mechanisms ensuring Freedom from Interference.
• Improved quality also for the non-safety-related software components which leads to a higher availability of the system. On the other hand, the following disadvantages have to be considered: • The development effort increases since all software elements have to be developed according to the highest ASIL. For the non-safety-related part an additional safety requirement is then applied, which requires the non-interference ("silence") with the safety-related part.
• As ASIL development does not mean that the software is error free, errors in these parts

are not prevented to propagate by design.
• Inclusion of third-party software (e. g. “blackbox” software) is more difficult, as the development process of these modules is often unknown or cannot be influenced.[8]

3.2.2 Mixed ASIL Design

A “Mixed ASIL Design” targets the development of software elements according to QM or a lower ASIL without jeopardizing the integrity of the entire software system, which may have a higher ASIL.
It may also enable the containment of errors in a partition. This concept requires a suitable software design on application level, i. e. functional blocks must be coherent and unwanted interlinking between functional blocks (e. g. via global variables) should be avoided.
It also requires a safety mechanism realizing the freedom from interference on hardware and software level which ensures that a software element with a lower ASIL cannot interfere with a software element with a higher ASIL.
This mechanism must be able to either prevent that a malfunction of one element leads to the malfunction of another element, or it must be able to detect such interference and to mitigate the effects in time.
This safety mechanism has to be developed according to the “Maximum ASIL” of the software safety requirements realized on this ECU. ISO 26262 mentions different aspects of possible interferences:
1. Memory, which includes the RAM as well as the CPU registers.
2.Timing and executions, which refers to blocking of execution, deadlocks and livelocks or the incorrect allocation of execution time in general.
3. Communication, summarizing all possible errors that could occur in the communication between software elements both within the ECU and across ECU boundaries.[8]

3.3 Freedom From Interefnece

"Freedom from interference" within ISO 26262 is a key concept that ensures the safe coexistence of safety-critical and non-safety-critical software components within a single automotive system, such as an Electronic Control Unit (ECU).
It refers to the requirement that the operation of non-safety-related software must not adversely affect the function or performance of safety-critical software.
This separation is crucial to prevent unintended interactions or faults in non-safety systems, like infotainment or convenience features, from propagating to safety-critical systems, such as braking or steering control.
Achieving freedom from interference involves implementing robust partitioning techniques, such as hardware and software isolation, memory protection, and controlled communication channels, to ensure that resources like CPU time, memory, and I/O operations are

allocated in a way that prevents faults from affecting safety-critical functions.
ISO 26262 defines several areas of interference that must be considered, including data, timing, control flow, and hardware resource interference. Ensuring freedom from interference is vital for maintaining functional safety and adhering to the standard's requirements, thereby enhancing the reliability and safety of increasingly integrated automotive systems. [9]

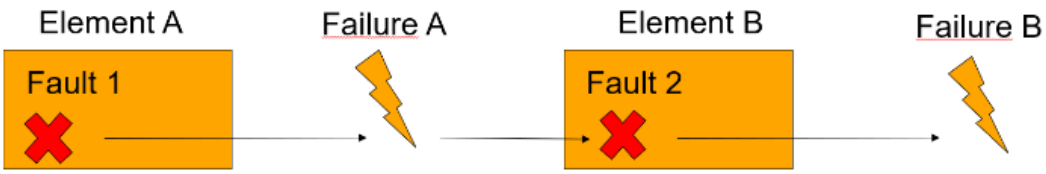

Figure 7: Cascading error problem

What that means is we are trying to avoid a dominio effect where the failure of one element would propagate in the system and leads to multiple failures. TYPES OF INTERFERENCE
ISO 26262 specifies three types of interference:

- timing and execution.
- memory.
- exchange of information.

Interference in the time domain occurs when execution of a safety-relevant software element is blocked due to a fault in another software element. This type of problem is detected and handled by what is called a watchdog - a combination of hardware and software, often supported through the operating system (RTOS). Once an error is detected the RTOS tries to recover the system or bring the system to a safe state

Figure 8: Interference in timing and execution.

Interference's in the memory space domain occurs when a software element accesses or modifies code or data belonging to another software element. This type of interference is related to corruption of memory content and device configuration data. Such interferences

are typically detected at run-time by a hardware component called the Memory Protection Unit (MPU). Once an error is detected, the MPU triggers an interrupt and the fault handler tries to restart all software elements located in the partition that caused the interrupt to occur.

Figure 9: Interference in shared memory space. [?]

There are several advantages that such integrated architecture provides, including:

- More reliable and efficient designs are possible with multiple components easily communicating with each other on a single ECU. **??**.
- Reduced material costs with less ECUs, wires, and connectors required in the design. **??**.
- Reduced potential for hardware failures with less transference of information between ECUs. **??**.

However, there are also risks associated with such integrated mixed criticality systems, such as:

- Increased likelihood that fault in one software element could cause another safety **??**.
- Added complexity in assuring FFI due to multi-core architectures. **??**.
- Higher chance of interference between software elements with shared global memory. **??**.

3.4 ISO 26262 Safety lifecycle

The ISO 26262 safety lifecycle outlines a comprehensive, structured process for ensuring functional safety throughout the entire lifecycle of an automotive system, from concept to decommissioning. The lifecycle is divided into several phases, each of which addresses different aspects of safety to ensure that the vehicle's electrical and electronic systems operate reliably under all conditions.

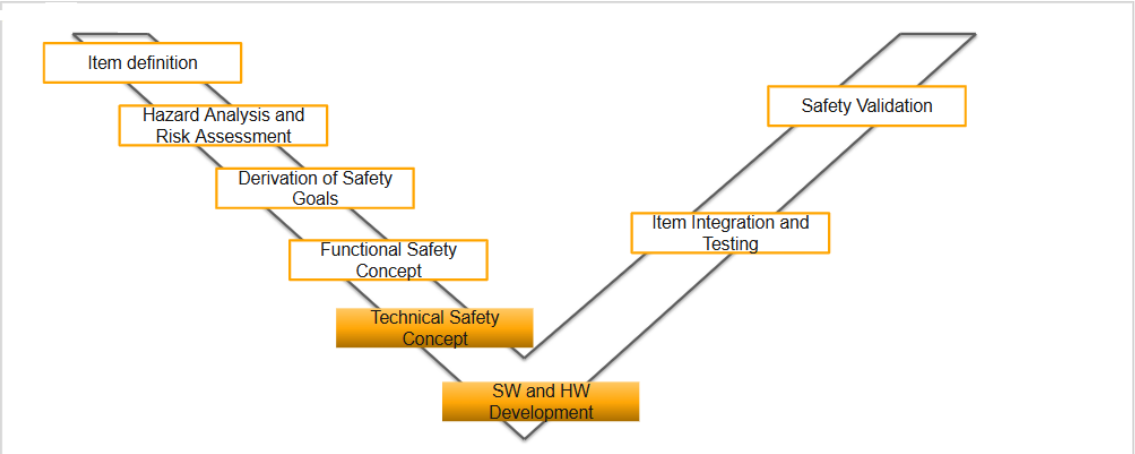

Figure 10: ISO 26262 Safety lifecycle.

3.5 Technical Safety Concept

The Technical Safety Concept (TSC) within ISO 26262 is a critical component in the development of automotive safety systems, providing a detailed framework for translating high-level safety goals into concrete technical requirements.
Derived from the Functional Safety Concept (FSC), the TSC focuses on defining the technical measures and safety mechanisms required to prevent or mitigate potential hazards identified in the hazard analysis and risk assessment (HARA) phase.
This includes specifying requirements for system architecture, hardware, and software that ensure the system's ability to detect, control, or tolerate faults without leading to unsafe conditions.
The TSC encompasses various aspects such as fault detection, error handling, redundancy, and fail-safe strategies to achieve the desired ASILs.
By establishing a clear set of technical safety requirements, the TSC ensures that all safety-related functions are consistently and rigorously implemented, verified, and validated throughout the system's development, thereby supporting the overall safety objectives defined by ISO 26262.

3.6 AUTOSAR

AUTOSAR (AUTomotive Open System ARchitecture) is an open and standardized automotive software architecture developed by a consortium of car manufacturers, suppliers, and tool developers. Established in 2003, AUTOSAR aims to create a common platform for the development of automotive software that enhances standardization, modularity, and scalability, and facilitates the reuse of software components across different vehicle platforms and manufacturers. As vehicles become increasingly software-driven, with functionalities ranging from infotainment systems to advanced driver-assistance systems (ADAS), AUTOSAR provides a robust framework that addresses the growing complexity of automotive software and electronics. [10]

3.6.1 AUTOSAR Architecture

The AUTOSAR architecture is designed to be modular and layered, ensuring that different software components can be independently developed, tested, and integrated. The architecture is divided into three main layers:

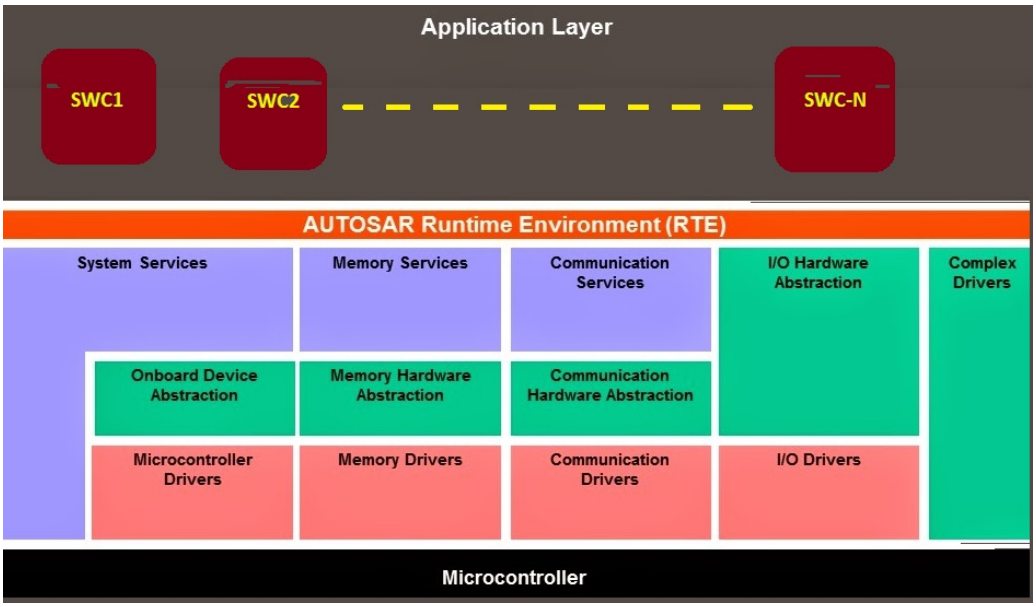

Figure 11: AUTOSAR Architecture diagram
[10]

Application Layer: This layer contains the application software components that execute specific functionalities, such as braking, steering, or climate control. Each application component is designed to be independent, facilitating the reuse of software across different vehicle platforms. These components communicate with each other and with the underlying layers via standardized interfaces defined by AUTOSAR, enabling interoperability and integration.

Runtime Environment (RTE): The RTE acts as middleware between the application layer and the basic software layer. It provides a standardized communication mechanism, ensuring that software components can exchange data regardless of their location (whether on the same ECU or different ECUs). The RTE abstracts the hardware details, allowing application software components to be hardware-agnostic. This layer plays a crucial role in enabling the portability of software across different hardware platforms.

Basic Software Layer: This foundational layer consists of standardized software modules that provide essential services required by the application components. It is further subdivided into several sub-layers:

Microcontroller Abstraction Layer (MCAL): Provides interfaces to the microcontroller peripherals and devices, abstracting the hardware specifics from the upper layers. ECU Abstraction Layer: Offers a unified interface to access peripherals and devices attached to the ECU, regardless of their hardware-specific implementation.

Services Layer: Provides generic services such as diagnostic services, memory management, communication services, and operating system functionalities. This layer ensures that critical functions like data transmission, error detection, and memory access are handled consistently across different ECUs.[10]

3.6.2 Advantages of AUTOSAR

Standardization and Interoperability: By providing a common platform and standardized interfaces, AUTOSAR facilitates the development of software that is compatible across different ECUs and vehicle platforms. This reduces integration efforts and ensures seamless interoperability between components from different suppliers.

Modularity and Scalability: AUTOSAR's modular architecture allows developers to create reusable software components, which can be easily integrated into different projects. This modularity not only accelerates development but also enhances scalability, making it easier to add or modify functionalities as needed.

Reduced Development Costs and Time: The reuse of standardized software components and the reduction in integration efforts result in significant cost savings. Furthermore, the ability to port software across different hardware platforms without major modifications reduces development time.[11]

Support for Functional Safety: AUTOSAR is designed to support safety-critical applications in compliance with standards such as ISO 26262. Its architecture facilitates the separation of safety-critical and non-safety-critical components, ensuring that faults in one part of the system do not compromise the overall system safety.

Improved Software Quality and Maintainability: The standardization of software development processes and tools improves the overall quality and maintainability of automotive software. Developers can focus on creating high-quality, application-specific functionality without worrying about low-level hardware integration.

3.6.3 Disadvantages of AUTOSAR

Complexity and Learning Curve: The AUTOSAR architecture is inherently complex due to its layered and modular nature. For organizations and developers new to AUTOSAR, there is a steep learning curve associated with understanding and implementing

the standard. This can lead to increased initial investment in terms of training and tool acquisition.[12]

Limited Flexibility for Custom Solutions: While AUTOSAR promotes standardization, it can also limit the flexibility to implement custom or proprietary features. This may hinder innovation in certain areas where a unique or tailored solution is required.

High Initial Development Costs: The adoption of AUTOSAR involves significant initial development costs, including the need for specialized tools, training, and infrastructure. This can be a barrier for smaller companies or projects with limited budgets.

Performance Overhead: The abstraction layers and modularity of AUTOSAR, while beneficial for standardization and portability, can introduce some performance overhead. This is particularly relevant for real-time applications where every millisecond of processing time is critical.

3.6.4 Applications of AUTOSAR within Functional Safety

AUTOSAR plays a pivotal role in the development of automotive systems that must comply with functional safety standards such as ISO 26262. In safety-critical systems—such as those involving braking, steering, or airbag deployment—faults can lead to hazardous situations. AUTOSAR addresses these challenges through several key features:

Freedom from Interference: AUTOSAR supports the concept of "freedom from interference," which is essential in mixed-criticality systems. This ensures that a fault in a non-safety-critical component does not affect the operation of safety-critical components. The use of the Memory Protection Unit (MPU) and partitioning mechanisms within AUTOSAR allows for strict separation of memory and resources, safeguarding safety-critical operations.

Error Detection and Fault Tolerance: AUTOSAR includes mechanisms for error detection, reporting, and handling, which are crucial for functional safety. The architecture supports various safety mechanisms, such as watchdogs and diagnostic services, which monitor the health of the system and take corrective actions in the event of a fault.

ASIL (Automotive Safety Integrity Level) Compliance: AUTOSAR helps in achieving compliance with different ASIL levels by providing guidelines and tools for the development and validation of software components according to their safety criticality. This ensures that all components meet the necessary safety requirements, minimizing the risk of system failure.

Support for Safe State Management: AUTOSAR provides mechanisms for transitioning the system to a safe state in the event of a detected fault. This is critical for ensuring that, even in the presence of faults, the vehicle can be brought to a safe condition without compromising the safety of passengers and other road users.

3.7 Software and Hardware development

Within ISO 26262, the development of both software and hardware plays a pivotal role in ensuring the functional safety of automotive systems.
The standard provides specific guidelines for the development processes of both software and hardware components to meet the required safety levels. For hardware, ISO 26262 Part 5 focuses on defining safety requirements and performing hardware design, analysis, and verification to ensure that all components, such as microcontrollers and sensors, are designed to handle potential faults and maintain safe operation.
It includes methods for calculating hardware metrics like failure rates, fault detection coverage, and diagnostic capabilities to achieve the desired Automotive Safety Integrity Levels (ASILs).
For software, ISO 26262 Part 6 provides detailed requirements for the entire software development lifecycle, from requirements specification and architectural design to coding, integration, testing, and verification.
The standard emphasizes the use of safe coding practices, robust design techniques, and comprehensive testing strategies to ensure that software components can detect and handle faults, avoid unintended interactions, and maintain their intended functionality under all conditions.
By providing a structured approach to both hardware and software development, ISO 26262 ensures that automotive systems achieve and maintain the highest levels of safety throughout their operational lifecycle.

3.7.1 AUTOSAR Based Solutions

As we mentioned before, Freedom from interference of software components is defined by three properties:

- Safe memory accesses
- Correct time execution
- Safe data exchange

A component's freedom from interference can be verified by classic verification measures, e.g. by code reviews. There are also approaches in which a specially developed code checker is used to check the freedom from interference of the basic software. Other measures may be taken in the software to ensure protection against hardware-related disturbances as well.

Safe memory access

A modern AUTOSAR operating system offers protection against faulty overwriting of memory contents. The protection is achieved by partitioning each functional group into a so-called OS application. Each OS application's data are allocated in separate memory partitions. Along with the application data, context-related data such as stacks and the contents of important registers are also located in such a memory partition. Access to these memory partitions is prevented by a Memory Protection Unit (MPU), which is part of the microprocessor hardware.
When switching the running task or the Interrupt Service Routine, the operating system executes a context switch. Here, the context data is stored, and the MPU is recon figured so that it only enables the memory partition for the task or Interrupt Service Routine that is active after the switch. This switch is only executed by the operating system and is safety-related.

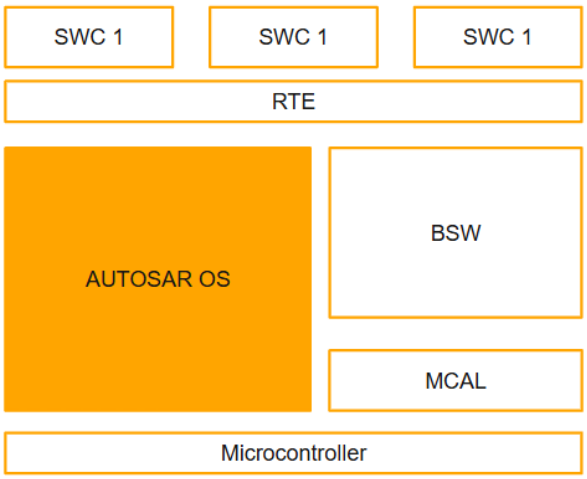

Figure 12: AUTOSAR OS within AUTOSAR Architecture.

It is important to monitor the program flow in safety-related systems. The Watchdog Manager specified in AUTOSAR is used for this purpose. This module, which supplements As the name suggests, this component controls the hardware watchdog, and it safely ensures a reset of the ECU in case of error. In addi tion, this component monitors for correct time flow of the application's tasks. Developers can set a number of parameters for the monitoring such as program flow, cycle times, minimum/maximum execution times, etc.

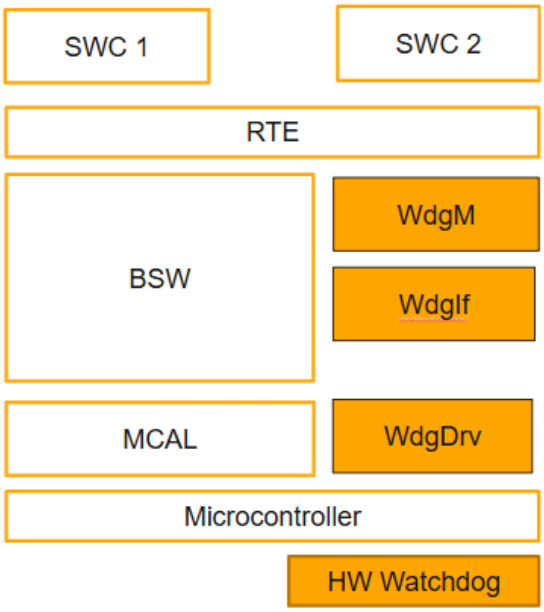

Figure 13: AUTOSAR Watchdog.

The main functions of the watchdog timer are

- ECU Reset in case of timeout**??**.
- Monitor the excution of program flow**??**.
- Monitor the minimun/maximum excuetion time of tasks**??**.

The third requirement for freedom from interference, which is implementation of reliable communication, is fulfilled by end-to-end protection . protects the data to be transmitted using a CRC and sequential message numbers. Strictly speaking, this does not ensure safe communication, rather just "integrity" in communication. Software cannot protect against data failure due to hardware errors, e.g. a bus line break. To ensure safe communication, additional measures must be taken in the hardware, e.g. in the form of redundant buses.

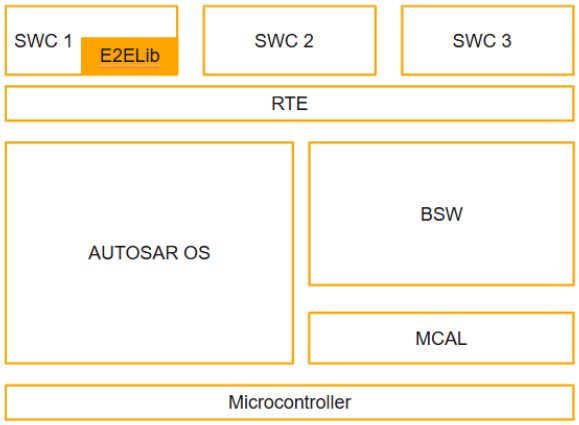

Figure 14: E2ELib within AUTOSAR Architecture

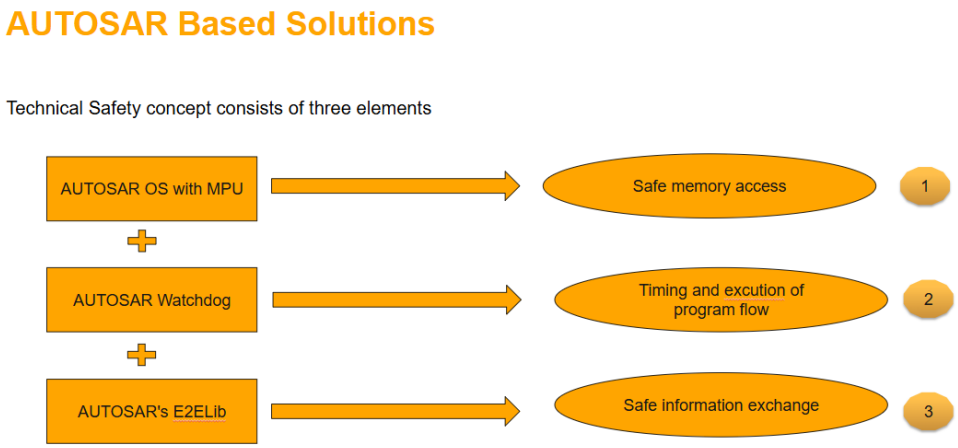

Figure 15: Technical concept of Autosar based solution.

How the components will work together

- Functions are grouped into separate OS Application
- Each OS Application is assigned a memroy partition
- Each OS application data + Context related data are stored in memory partition
- Access to these Memory partitions is Protected by MPU
- MPU is reconfigured at Runtime to enable the running task to access it‘s memory partition +

3.7.2 MPU Only Based Solutions

An MPU (Memory Protection Unit) only-based solution for achieving freedom from interference with Locator for ASIL (Automotive Safety Integrity Level) partitioning, coupled with a static MPU configuration, offers a practical and efficient method for managing mixed-criticality software systems within a single automotive ECU. In this approach, the MPU is configured to create fixed memory regions with predefined access rights, isolating safety-critical and non-safety-critical software components based on their ASIL levels.
The static configuration simplifies the system design by avoiding the overhead and complexity associated with dynamic memory management, making it particularly suitable for embedded environments where deterministic behavior and minimal runtime overhead are essential.
Locator plays a crucial role in this setup by strategically allocating and partitioning memory segments to align with the safety requirements, ensuring that each ASIL partition is adequately protected from unauthorized access. This method ensures that any faults or unexpected behaviors in non-safety-critical modules, such as infotainment systems, do not compromise the integrity or performance of safety-critical functions, such as airbag deployment or braking systems.
By leveraging the MPU's hardware-based memory protection capabilities and a static configuration, automotive developers can achieve robust safety compliance per ISO 26262 standards while maintaining efficient resource utilization on the ECU.

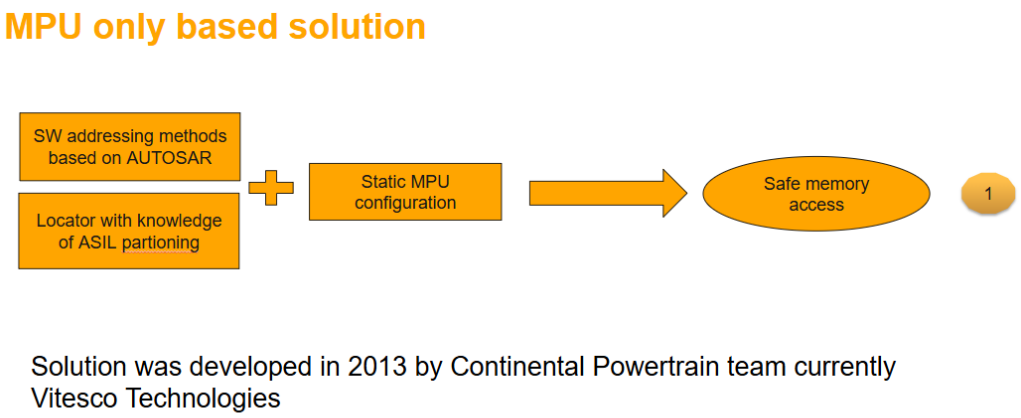

Figure 16: Technical concept of MPU only based solution.

AUTOSAR addressing methods

Are a set of rules defined by the AUTOSAR standard to allocate variables in memory in a way that supports both safety and efficiency requirements for automotive ECUs. These methods are essential when implementing hardware memory protection, which is crucial for achieving freedom from interference among software components of different

criticality levels. AUTOSAR specifies that internal variables—those which are integral to the function of a software module and need to be protected against unauthorized access or modification—should be mapped into protected memory areas. This ensures that critical data is shielded from any unintended interference, thereby preserving system integrity and safety, especially in safety-critical applications that adhere to ISO 26262 standards. Conversely, buffers used for data exchange between modules, which do not require stringent protection, are mapped into unprotected memory areas. This separation allows for more efficient data sharing and communication between modules without compromising the safety-critical variables. By using these addressing methods, AUTOSAR ensures that the memory is allocated in a structured and secure manner, balancing the need for both protection and performance in mixed-criticality environments within automotive ECUs.

3.8 Tools and technologies

This section discusses various tools and technologies that were studied and surveyed which might be useful for the implementation of the concept formulated under this master thesis.
Each tool and technology is understood from the point of present research work and how can the same be utilized. The explanation to the usage of each is covered under their respective topics.

3.8.1 The AURIX Development Studio

is an integrated development environment (IDE) specifically designed for developing applications on Infineon's AURIX family of microcontrollers, which are widely used in automotive and industrial applications.
This development platform provides a comprehensive suite of tools and features to facilitate the development, debugging, and testing of both safety-critical and non-safety-critical software, in compliance with standards such as ISO 26262.
The AURIX Development Studio offers a user-friendly interface with support for various programming languages, including C and C++, and includes a powerful compiler, linker, and debugger that are optimized for the AURIX architecture.
It also integrates advanced features such as real-time trace and analysis, code generation for peripheral configuration, and multi-core programming support, enabling developers to efficiently manage the complexity of multi-core microcontroller projects.
The studio supports seamless integration with third-party tools and provides comprehensive documentation and code examples, making it an ideal environment for accelerating development cycles and improving the reliability and safety of automotive applications.

3.8.2 Da Vinci Configurator

Da Vinci Configurator is a powerful tool from Vector Informatik, designed to support the development of automotive software in compliance with the AUTOSAR (AUTomotive

Open System ARchitecture) standard. As the automotive industry continues to advance towards more complex, software-defined vehicles, the need for efficient and reliable software configuration tools has become critical. Da Vinci Configurator addresses this need by providing a comprehensive environment for configuring, generating, and integrating AUTOSAR basic software modules and the Runtime Environment (RTE).

Key Features and Functionality Da Vinci Configurator is specifically tailored for developers working with AUTOSAR-based systems. It offers a range of features that streamline the development process:

Comprehensive Configuration Management: Da Vinci Configurator provides a graphical interface for the configuration of AUTOSAR basic software modules, including the operating system (OS), memory management, communication stacks (CAN, LIN, FlexRay, Ethernet), and diagnostic services. This graphical approach simplifies the often complex task of configuring multiple software components to ensure they work seamlessly together.

Support for AUTOSAR Standards: The tool is regularly updated to support the latest versions of the AUTOSAR standard, ensuring compatibility and future-proofing software development projects. This includes support for both AUTOSAR Classic and Adaptive platforms, making Da Vinci Configurator a versatile choice for a wide range of applications.

Efficient Code Generation: Da Vinci Configurator automates the generation of AUTOSAR-compatible C code for the configured modules. This reduces manual coding effort and minimizes the risk of human error, significantly speeding up the development cycle. The generated code is optimized for performance and memory usage, which is crucial in the resource-constrained environments typical of automotive ECUs.

Seamless Integration with Other Vector Tools: The configurator is designed to integrate seamlessly with other tools in the Vector Da Vinci suite, such as Da Vinci Developer, which focuses on the application layer of AUTOSAR. This integration allows for a smooth workflow from configuration through to development and testing, enhancing overall productivity and reducing time-to-market.

Advanced Validation and Diagnostics: Da Vinci Configurator includes built-in validation and diagnostic capabilities that check configurations for consistency, completeness, and compliance with AUTOSAR standards. This helps developers identify and correct issues early in the development process, improving software quality and reducing the likelihood of costly errors downstream.

Advantages of Using Da Vinci Configurator The Da Vinci Configurator offers several significant advantages to automotive software developers:

Reduced Development Time and Effort: By automating the configuration and code generation processes, Da Vinci Configurator significantly reduces the time and effort required to develop AUTOSAR-based software. This allows teams to focus more on innovation and functionality rather than on routine coding tasks.

3.8.3 C programming language

The C programming language is a cornerstone of embedded systems development due to its efficiency, control, and versatility.
Renowned for its close-to-hardware capabilities, C allows developers to write software that directly interacts with hardware components, making it ideal for resource-constrained environments typical in embedded systems.

Its low-level access to memory and system registers, combined with a relatively straightforward syntax, enables precise manipulation of hardware and optimization of performance. C's efficiency in terms of execution speed and memory usage is critical for embedded systems, where resources are often limited and real-time performance is essential.
Furthermore, C's portability across various hardware platforms and its extensive ecosystem of libraries and tools support the development of robust, maintainable, and scalable embedded applications.
Its widespread use in embedded systems is also bolstered by a large community and extensive documentation, which provide valuable resources for tackling complex development challenges. Overall, C remains a preferred language in embedded systems for its balance of control, performance, and flexibility.

4 Concept for FFI with MPU

4.1 Memory Protection Unit

A Memory Protection Unit (MPU) is a hardware component integrated into microcontrollers and processors, primarily used to enhance the security and reliability of embedded systems by managing and protecting memory access. The MPU enables the division of memory into distinct regions, each with specific access permissions (such as read, write, or execute) and attributes (such as cacheability or bufferability). This allows the system to enforce access control policies, ensuring that different software modules or tasks only access the memory regions they are authorized to, thereby preventing unintended or malicious access to critical data and code. The use of an MPU is particularly important in systems with mixed-criticality, such as automotive ECUs, where safety-critical functions must be isolated from non-safety-critical ones to maintain system integrity and comply with safety standards like ISO 26262. By statically or dynamically configuring the MPU to protect safety-critical code and data from interference by lower-criticality tasks, developers can ensure that faults or bugs in non-critical software components do not propagate and affect the operation of critical safety functions. This hardware-based memory protection mechanism is essential for achieving robustness, security, and functional safety in modern embedded systems.[13]

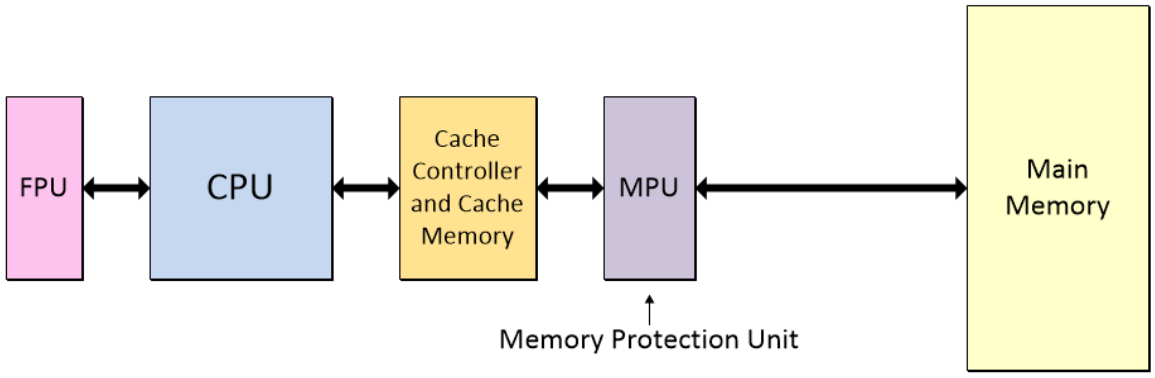

Figure 17: MPU Diagram

4.2 Hardware Architecture

The AURIX™ TCxx4 Lite Kit is a development platform designed by Infineon Technologies, specifically for evaluating and developing applications on the AURIX™ TCxx4 family of microcontrollers.
These microcontrollers are based on the TriCore™ architecture, which combines a powerful single-core performance with the capabilities required for automotive and industrial applications. The Lite Kit provides an entry-level, cost-effective solution for developers to experiment with and prototype automotive and safety-critical applications, such as

powertrain, chassis, and advanced driver-assistance systems (ADAS).

The AURIX™ TCxx4 Lite Kit includes essential components to start development, such as onboard debugging capabilities, a comprehensive set of peripherals, and connectivity options.

It supports the full range of development tools compatible with AURIX™, including the AURIX Development Studio, enabling developers to write, compile, and debug code efficiently. The kit also provides access to various interfaces like CAN, SPI, and UART, allowing developers to connect and test their applications in a simulated environment. With its integrated features and compatibility with safety standards like ISO 26262, the AURIX™ TCxx4 Lite Kit is ideal for both beginners and experienced engineers looking to create reliable, safe, and high-performance automotive applications.

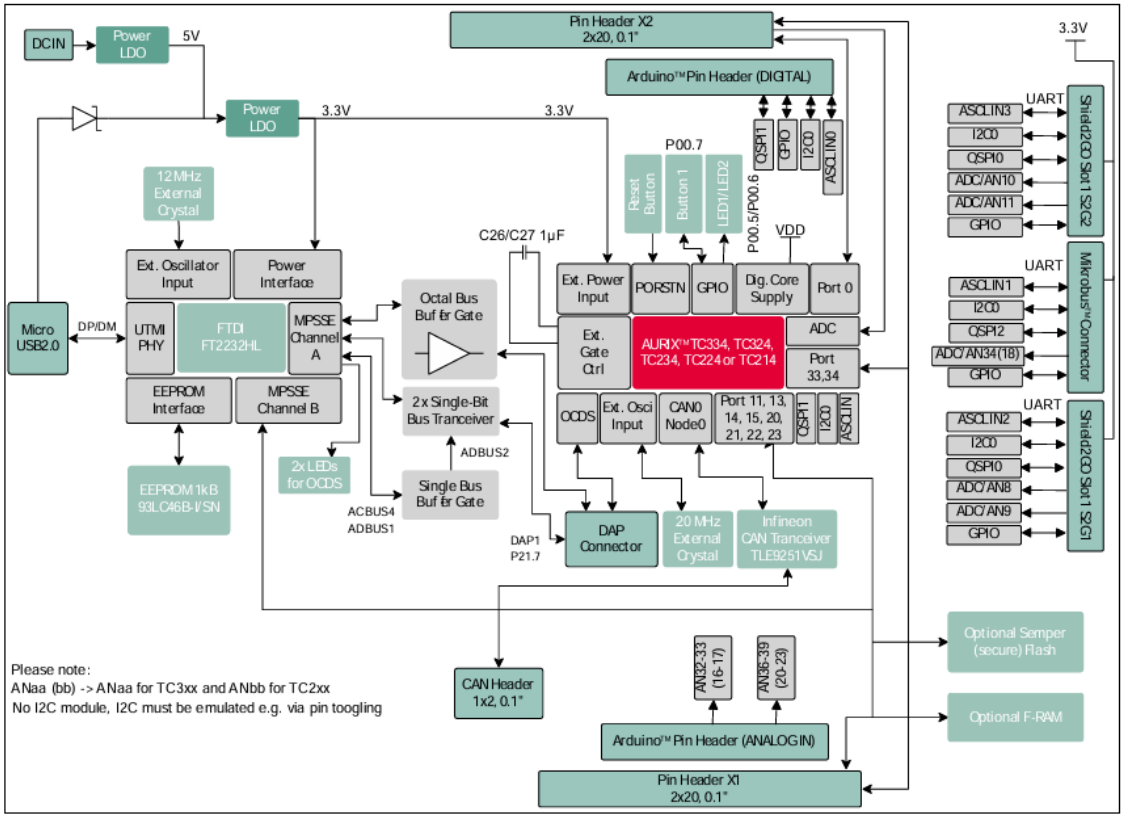

Figure 1 **Block Diagram of the AURIX™ TCxx4 lite Kit**

Figure 18: Block diagram of the AURIX TCxx4 lite kit. [14]

4.3 TriCore Architecture

The TriCore architecture, developed by Infineon Technologies, is a high-performance microcontroller platform that has gained significant traction in the automotive industry,

particularly for functional safety applications. Combining a RISC (Reduced Instruction Set Computing) processor core with DSP (Digital Signal Processing) capabilities and a set of powerful peripherals, TriCore microcontrollers are designed to meet the rigorous demands of modern automotive systems, where safety, real-time performance, and efficiency are paramount.

Overview of TriCore Architecture At its core, the TriCore architecture is a unified architecture that blends the strengths of different processing paradigms. It integrates a 32-bit microcontroller with high processing power, enabling efficient handling of both control and data-intensive tasks. This combination makes it an ideal choice for applications that require high computational capabilities, such as engine control, transmission control, and advanced driver-assistance systems (ADAS). [14]

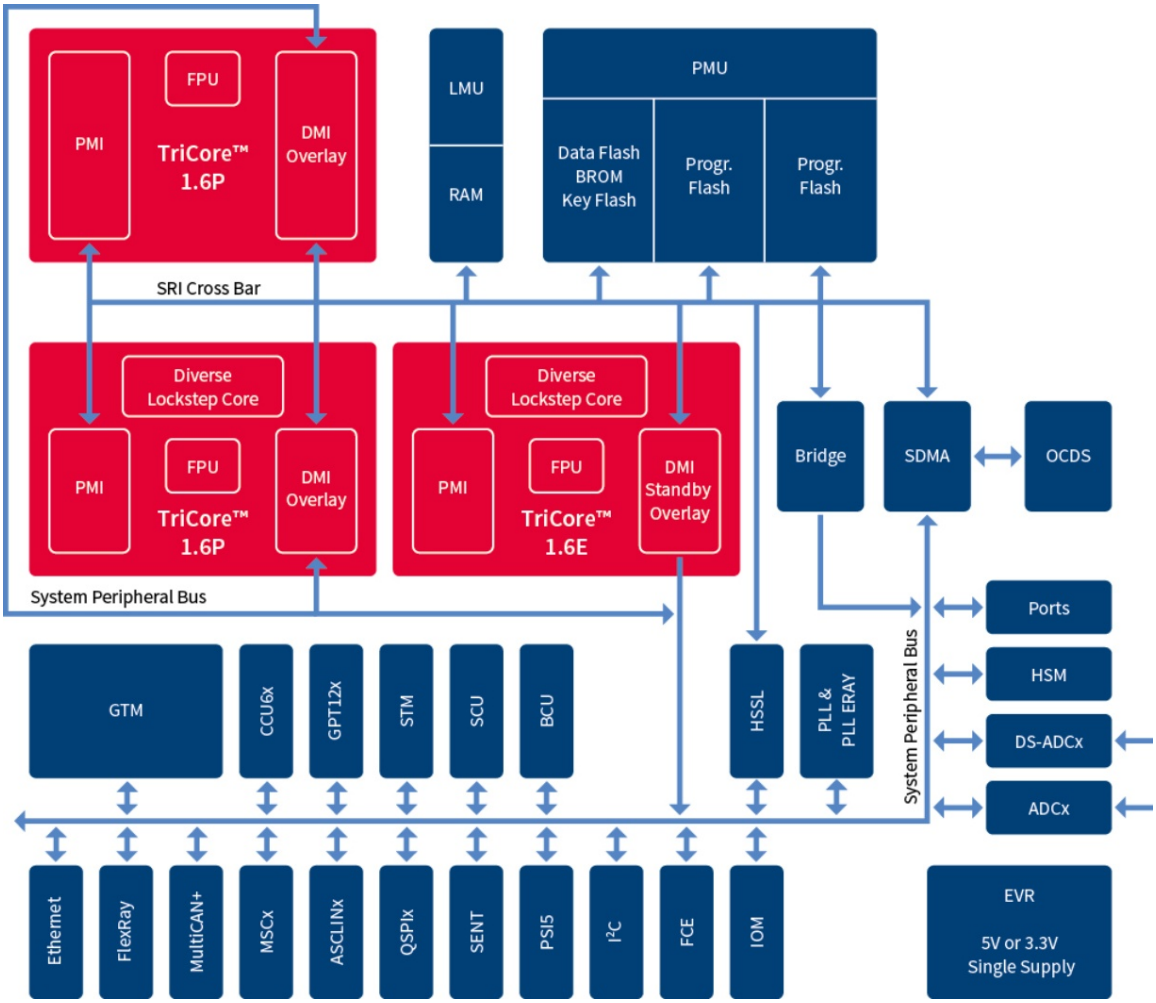

Figure 19: TriCore Architecture

The architecture's distinctive feature is its ability to simultaneously handle multiple computational workloads through a single processor core. This capability is achieved by integrating RISC, DSP, and microcontroller (MCU) functionalities. The RISC component

ensures efficient execution of general-purpose code, the DSP capabilities handle complex mathematical operations, and the MCU features facilitate real-time control tasks. This multifaceted design approach allows TriCore to deliver superior performance and flexibility, which is critical for modern automotive applications.

Functional Safety in Automotive Systems Functional safety is a critical aspect of automotive design, particularly with the increasing adoption of autonomous driving technologies and the growing complexity of electronic control systems. Functional safety ensures that a system behaves predictably and safely in response to its inputs, even in the presence of faults. In the automotive context, this often relates to ISO 26262, the international standard for the functional safety of electrical and electronic systems in production vehicles.

TriCore architecture plays a pivotal role in achieving functional safety in automotive projects. Infineon's TriCore microcontrollers are designed with numerous safety mechanisms that help detect and mitigate faults, thereby ensuring the system's safe operation. These safety features include lockstep cores, memory protection units, error correction codes (ECC) for memories, and hardware redundancy. The lockstep architecture, for instance, involves running two processor cores in parallel and comparing their outputs to detect faults, providing a critical layer of error detection.

Usage of TriCore in Automotive Functional Safety Projects In automotive functional safety projects, the use of TriCore architecture is widespread due to its high reliability and advanced safety features. It is used extensively in electronic control units (ECUs) for powertrain, chassis, and body electronics. These ECUs require not only real-time processing capabilities to handle the continuous flow of data from sensors and control devices but also robust safety mechanisms to ensure the system remains safe and operational under all conditions.

TriCore's integration of safety features directly supports the implementation of safety-critical functions, such as anti-lock braking systems (ABS), electronic stability control (ESC), airbag deployment systems, and electric power steering (EPS). The architecture's ability to execute complex algorithms quickly and reliably makes it ideal for these applications, where milliseconds can make a difference in ensuring passenger safety. [14]

Moreover, the TriCore architecture supports comprehensive safety software libraries and toolchains, enabling automotive developers to design, implement, and validate safety-related software more efficiently. Infineon also offers extensive documentation and support for ISO 26262 compliance, making it easier for automotive manufacturers to meet stringent safety standards.

Conclusion The TriCore architecture from Infineon is a powerful and versatile platform that is well-suited for functional safety applications in the automotive industry. Its unique

combination of high-performance computing, robust safety features, and real-time processing capabilities makes it an ideal choice for modern automotive systems that demand both performance and reliability. As the automotive industry continues to evolve towards more sophisticated and autonomous vehicles, the role of TriCore microcontrollers in ensuring functional safety will only become more crucial.

4.4 AURIX Protection system

One of the domains that TriCore supports is safety-critical embedded applications. The architecture features a protection system designed to protect core system functionality from the effects of software errors in less critical application tasks, and to prevent unauthorised tasks from accessing critical system peripherals.
The protection system also facilitates debugging. It detects and traps errors that might otherwise go unnoticed until it was too late to identify the cause of the error. The overall protection system is composed of four main subsystems:
1. A trap occurs as a result of an event such as a Non-Maskable Interrupt (NMI), an instruction exception, memory management exception or an illegal access. Traps are always active; they cannot be disabled by software action.

2. The I/O Privilege Level: TriCore supports three I/O modes: User-0 mode, User-1 mode and Supervisor mode. The User-1 mode allows application tasks to directly access non-critical system peripherals.
This allows embedded systems to be implemented efficiently, without the loss of security inherent in the common practice of running everything in Supervisor mode. (The default behaviour of the User-1 mode may be overriden by the system control register).

3. The Memory Protection System: This protection system provides control over which regions of memory a task is allowed to access, and what types of access it is permitted.

4. The Temporal Protection system. This protection system provides protection against run-time overrun.

For applications that require virtual memory, the optional Memory Management Unit (MMU) supports a familiar page-based model for memory protection. That model gives each memory page its own access permissions. The relatively conventional MMU design and the page-based memory protection model facilitate porting of standard operating systems that expect this model. [14]

4.4.1 Trap system

A trap occurs as a result of an event such as a Non-Maskable Interrupt (NMI), an instruction exception, memory management exception or an illegal access. Traps are always active; they cannot be disabled by software action. This chapter describes the different traps that can occur and the TriCore® architecture's trap handling mechanism. The TriCore architecture specifies eight general classes for traps. Each class has its own trap handler, accessed through a trap vector of 32 bytes per entry, indexed by the hardware-defined trap class number. Within each class, specific traps are distinguished by a Trap Identification Number (TIN) that is loaded by hardware into register D[15] before the first instruction of the trap handler is executed. The trap handler must test and branch on the value in D[15] to reach the subhandler for a specific TIN. Traps can be further classified as synchronous or asynchronous, and as hardware or software generated.

The trap classes are:

- MMU (Memory Management Unit)
- Internal Protection
- Instruction Error
- Context Management
- System Bus and Peripherals
- Assertion Trap
- System Call
- Non-Maskable Interrupt (NMI)

4.4.2 The I/O Privilege Level

Access Privilege Level Control (I/O Privilege) Software Managed Tasks (SMTs) are created through the services of a real-time kernel or Operating System, and are dispatched under the control of scheduling software. Interrupt Service Routines (ISRs) are dispatched by hardware in response to an interrupt. An ISR is the code that is invoked directly by the processor on receipt of an interrupt. SMTs are sometimes referred to as user tasks, assuming that they execute in User Mode.

- User-0 Mode: Used for tasks that do not access peripheral devices. This mode may not enable or disable interrupts.
- User-1 Mode: Used for tasks that access common, unprotected peripherals. Typically this would be a read or write access to serial port, a read access to timer, and most I/O status registers. Tasks in this mode may disable interrupts. (The default behaviour of this mode may be overriden by the system control register).

- Supervisor Mode: Permits read/write access to system registers and all peripheral devices. Tasks in this mode may disable interrupts.

A set of state elements are associated with any task, and these are known collectively as the task's context. The context is everything the processor needs to define the state of the associated task and enable its continued execution. This includes the CPU General Registers that the task uses, the task's Program Counter (PC), and its Program Status Information (PCXI and PSW). The architecture efficiently manages and maintains the context of the task through hardware. [14]

4.4.3 Memory Protection system

Provides control over which regions of memory a task is allowed to access, and what types of access is permitted.
• Range Based The range-based memory protection system is designed for small and low cost applications to provide coarse grained memory protection for systems that do not require virtual memory. This range-based system is detailed in this chapter.
• Page Based For applications that require virtual memory, the optional Memory Management Unit (MMU) supports a familiar model that gives each memory page its own access permissions. Effective Addresses Effective addresses are translated into physical addresses using one of two translation mechanisms:
• Direct translation.
• Page Table Entry (PTE) based translation (Optional MMU only). Memory protection for addresses that undergo direct address translation is enforced using the range-based memory protection system described in this chapter.

Range Based Memory Protection

The range-based memory protection system is designed for small and low cost applications to provide memory protection for systems that do not require virtual memory.

A Protection Range is a continuous part of address space for which access permissions may be specified. A Protection Range is defined by the Lower Boundary and the Upper Boundary. An address belongs to the range
if: • Lower Boundary <= Address < Upper Boundary
There are two groups of Protection Ranges:
• Data Protection Ranges specify data access permissions
• Code Protection Ranges specify instruction fetch permissions
The number of code and data protection ranges is implementation dependent, limited to a minimum of four and a maximum of 32 for each.
The granularity for lower and upper boundaries is 8-bytes for data protection ranges and 32-bytes for code protection ranges The three least significant bits of the Data Protection upper and lower bound registers are not writeable and always return zero.

The five least significant bits of the Code Protection upper and lower bound registers are not writeable and always return zero. [14]

Access Permissions

Access Permissions define the kind of access allowed to a protection range. The available types are:
• Data Read
• Data Write
• Instruction Fetch
Each access type can be separately permitted by setting the corresponding Access Flag.

Protection Sets A complete set of access permissions defined for the whole address space used, is called a Protection Set.

- A selection of Code Protection Ranges
- A selection of Data protection Ranges
- The Access Permissions defined for each Range
- A selection of execute enabled Code Protection Ranges
- A selection of write enabled Data protection Ranges
- A selection of read enabled Data protection Ranges

The Protection
Set defines both data access permissions and instruction fetch permissions.
In a Protection Set each data protection range has associated Read Enable and Write Enable flags. Each Code Protection Range has an associated Execution Enable flag. The number of memory protection sets provided is specific to each TriCore implementation, limited to a minimum of two and a maximum of eight.

Having multiple protection sets allows for a rapid change of the whole set of access permissions when switching between User and Supervisor mode, or between different User tasks. At any given time one of the sets is the current protection register set which determines the legality of memory accesses by the current task. The PSW.PRS field determines the current protection register set number.

4.4.4 Temporal Protection System

The TriCore™ Temporal Protection System is used to guard against run-time over-run. The system consists of two primary mechanisms, the temporal protection timers and the exception timers.

Temporal protection Timers

The temporal protection timers system consists of three independent decrementing 32 bit counters, arranged to generate a Temporal Asynchronous Exception (TAE) trap (Class-4, Tin-7), on decrement to zero.
The Temporal Protection System is enabled by setting the TPROTEN bit in the SYSCON register.
A timer is activated by writing a non-zero value to the TPS-TIMERx register.
After activation, the timer will decrement by one on each CPU clock cycle.
The timer will continue to decrement until either the count value reaches zero, or the timer is de-activated by writing zero to the TPS-TIMERx register. The current timer value can be read from the TPS-TIMERx register.
On a count decrement from one to zero, the associated TEXP bit in the TPS-CON register is set. The TEXP bit is cleared by any write to the associated TPS-TIMERx register.

On setting any TEXP bit in the TPS-CON register, the TTRAP bit in the same register is set.
A TAE trap is raised whenever the TTRAP bit transitions from zero to one.
The TTRAP bit is cleared by any write to the TPS-CON register. However attempting to clear the register while any TEXP bit is set will cause the TTRAP bit to be re-enabled and a new TAE trap is generated.
This ensures that no time-out event is missed during the handling of another TAE trap. [14]

4.5 Hardware

The AURIX™ TC375 Lite Kit is a versatile and cost-effective development platform designed by Infineon Technologies, built around the powerful AURIX TC375 microcontroller. This microcontroller is based on Infineon's TriCore™ architecture, which integrates a 32-bit microcontroller core with both high-performance processing and safety features, making it ideal for automotive, industrial, and safety-critical applications. The TC375 microcontroller includes three TriCore™ CPUs running at 300 MHz, along with hardware accelerators for secure cryptography and advanced data processing tasks. It also features a range of safety mechanisms, including memory protection, lockstep cores, and error correction, which makes it compliant with ISO 26262 standards for functional safety.

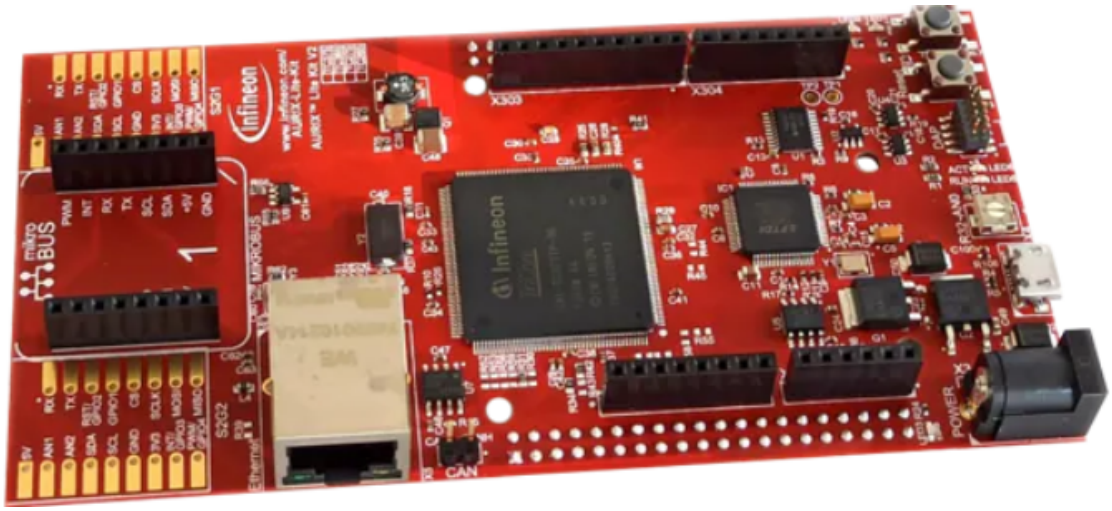

Figure 20: AURIX TC375 Lite Kit

The AURIX TC375 Lite Kit provides developers with an efficient platform to explore and experiment with this architecture. It offers various peripherals and interfaces, such as CAN-FD, Ethernet, and ADC channels, which allow for easy testing of real-time automotive applications like motor control, battery management, and advanced driver assistance systems (ADAS). The kit includes a range of onboard debugging and programming options, allowing developers to seamlessly load and test their applications using tools like Infineon's AURIX Development Studio or third-party IDEs such as Eclipse.

One of the standout features of the TC375 Lite Kit is its integration of safety and security features, which are critical in automotive and industrial applications. The hardware supports a wide range of development environments, with access to safety libraries and toolchains that simplify compliance with industry standards. The board's extensibility is supported by numerous GPIOs, making it adaptable to different projects and prototyping needs. Additionally, its compact size and affordability make it a perfect starting point for developers or teams interested in harnessing the capabilities of AURIX microcontrollers in a practical, hands-on way.

In summary, the AURIX TC375 Lite Kit is a highly flexible and robust development tool for engineers and researchers working on safety-critical and high-performance applications. Its TriCore architecture, combined with its advanced safety and security features, makes it ideal for automotive systems, while its ease of use ensures that developers can rapidly prototype and deploy sophisticated solutions across various industries.

4.6 Implementation

Each CPU has a Memory Protection Unit that can restrict what memory ranges are allowed for each master (e.g. DMA, CPU) based on a master ID In AURIX™ TC3xx, the

MPU supports:– 18 data ranges that specify which memory ranges a master is allowed to access for data. Read and write permissions can be specified for each range– 10 code ranges that specify which memory ranges the CPU is allowed to access for instructions In AURIX™ TC3xx, the CPU has 6 different protection sets (PRS) that specify which combination of data ranges and code ranges are active When the MPU is enabled, an instruction or data access outside of the specified MPU ranges selected by the active protection set immediately causes a CPU trap and optionally an alarm to the Safety Management Unit (SMU).

The CPU Memory Protection Unit is one of the many AURIX™ safety mechanisms that help to protect against random hardware faults as well as systematic faults

Protection Ranges are defined by a Lower Boundary and an Upper Boundary. An address belongs to the range if: – Lower Boundary <= Address < Upper Boundary The granularity of the memory protection ranges differ for data and code: – Data protection ranges have a granularity of 8 bytes – Code protection ranges have a granularity of 32 bytes The permission to access a memory location is the logical 'OR' of the memory range permissions. Where ranges intersect, the MPU grants the most permissive access

4.6.1 Configuration of the MPU

To enable memory protection, the following steps have to be followed:
Define the Data/Code Protection Ranges where access must be granted Define the type of access to grant in a defined Protection Set– Enable read and/or write in case of definition of a Data Protection Range (DPR)– Enable code execution in case of definition of a Code Protection Range (CPR)
Select the active Protection Set

Enable the memory protection

4.6.2 Definition of Data Protection Ranges

The definition of a Data Protection Range is done through the function define-data-protection-range(), which sets the lower and upper boundaries of the given CPU Data Protection Range number.
The lower and upper boundaries are set by calling the intrinsic function mtcr(), which moves contents of a data register to the addressed Core Special Function Register (CSFR). mtcr() performs a Move To Core Register (MTCR) TriCore™ instruction and is followed by an ISYNC instruction. The ISYNC instruction ensures that the effects of the Core Special Function Register (CSFR) update are correctly seen by all following instructions. An ISYNC must always be performed after an MTCR instruction.

4.6.3 Definition of Code Protection Ranges

The definition of a Code Protection Range is done through the function define-code-protection-range(), which sets the lower and upper boundaries of the given CPU Code Protection Range number.

The lower and upper boundaries are set by calling the intrinsic function mtcr(). Configuration of Read Access for a Data Protection Range The read access to a Data Protection Range (DPR) is provided with the function enable-data-read(), which enables the read access to the given DPR on the addressed Protection Set.
To enable the read access, first the CPU Data Protection Read Enable register (CPU-DPRE) is read with the intrinsic function mfcr(), the bit corresponding to the given DPR is set, and finally the modified value is stored back to the register with the intrinsic function mtcr() (essentially, a load-modify-store operation, which ensures that previously made changes are not overwritten).
mfcr() moves contents of the addressed Core Special Function Register (CSFR) into a data register by calling the TriCore™ instruction Move From Core Register (MFCR).

4.6.4 Configuration of Write Access for a Data Protection Range

The write access to a Data Protection Range is provided with the function enable-data-write(), which enables the write access to the given DPR on the addressed Protection Set.

To enable the write access, first the CPU Data Protection Write Enable register (CPU-WPRE) is read with the intrinsic function mfcr(), the bit corresponding to the given DPR is set, and finally the modified value is stored back to the register with the intrinsic function mtcr().

Configuration of Code Execution Access for a Code Protection Range The code execution access to a Code Protection Range (CPR) is provided with the function enable code execution(), which enables the code execution access of the given CPR on the addressed Protection Set.
To enable code execution access, first the CPU Code Protection Execute Enable register (CPU CPXE) is read with the intrinsic function mfcr(), the bit corresponding to the given CPR is set, and finally the modified value is stored back to the register with the intrinsic function mtcr().

4.6.5 Selection of the Active Protection Set

The active protection set is selected with the function set active protection set(), which sets the PRS bitfield of the Program Status Word (PSW) register accordingly to the given parameter. The set active protection set() function needs to be declared as inline because

the PSW is one of the registers automatically saved to the Context Save Area (CSA) when a function is called.
If this function was not declared as inline, the Upper Context (16 registers including the PSW) would be automatically saved to the CSA and re-loaded when the function returns, thus losing the change to the PSW. By default, the active Protection set is the Protection Set 0.

4.6.6 Activation of Memory Protection

The memory protection is enabled with the function enable memory protection(), which turns on the memory protection by setting the PROTEN bitfield of the System Configuration (SYSCON) register.

5 Results and Future Work

5.1 Results

A Memory Protection Unit (MPU) is a hardware component integrated into microcontrollers and processors, primarily used to enhance the security and reliability of embedded systems by managing and protecting memory access. The MPU enables the division of memory into distinct regions, each with specific access permissions (such as read, write, or execute) and attributes (such as cacheability or bufferability). This allows the system to enforce access control policies, ensuring that different software modules or tasks only access the memory regions they are authorized to, thereby preventing unintended or malicious access to critical data and code. The use of an MPU is particularly important in systems with mixed-criticality, such as automotive ECUs, where safety-critical functions must be isolated from non-safety-critical ones to maintain system integrity and comply with safety standards like ISO 26262. By statically or dynamically configuring the MPU to protect safety-critical code and data from interference by lower-criticality tasks, developers can ensure that faults or bugs in non-critical software components do not propagate and affect the operation of critical safety functions. This hardware-based memory protection mechanism is essential for achieving robustness, security, and functional safety in modern embedded systems.

5.2 Future work

Multiple opportunities are available for future work such as

Integrating a GUI into the Product Having a Graphical User Interface (GUI) will make the working with the configuration much easier and will eliminate the time needed for the data-sheet studying.

ARM Architecture design Another popular architecture within the automotive world is the ARM architecture and developing the same solution with ARM architecture will be an additional benefits that will make the solution fits into more projects

Hardware abstraction layer Having a Hardware abstraction layer will make the solution indepenednt from the microcontroller architecture and will be a great addition and will reduce the development time needed.

Literature references

[1] World Health Organization. *Global status report on road safety 2018*. World Health Organization, 2019.

[2] Antonio Arena, Fabrizio Tronci, Ida Maria Savino, and Gianluca Dini. A case study in obtaining freedom from interference in a mixed-asil architecture. In *2023 Annual Reliability and Maintainability Symposium (RAMS)*, pages 1–7. IEEE, 2023.

[3] Fei Su, Prashant Goteti, and Min Zhang. On freedom from interference in mixed-criticality systems: A causal learning approach. In *2019 IEEE International Test Conference (ITC)*, pages 1–10. IEEE, 2019.

[4] H. Gomez and P. Martin. A novel approach to autonomous driving systems. Technical report, HAL, 2019.

[5] International Electrotechnical Commission. Iec 62304:2006 - medical device software – software life cycle processes. Technical report, International Electrotechnical Commission, 2006.

[6] Ron Bell. Introduction to iec 61508. In *Acm international conference proceeding series*, volume 162, pages 3–12. Citeseer, 2006.

[7] Hans-Leo Ross. *Functional Safety for Road Vehicles*. Springer Cham, 1 edition, 2016.

[8] ZVEI. Software for safety-related automotive systems: Best practice guideline. Technical report, ZVEI, August 2016. Accessed: 2024-09-04.

[9] Tasking. Freedom from interference: Part 1. Technical report, Tasking, February 2021. Accessed: 2024-09-04.

[10] Simon Fürst, Jürgen Mössinger, Stefan Bunzel, Thomas Weber, Frank Kirschke-Biller, Peter Heitkämper, Gerulf Kinkelin, Kenji Nishikawa, and Klaus Lange. Autosar–a worldwide standard is on the road. In *14th International VDI Congress Electronic Systems for Vehicles, Baden-Baden*, volume 62. Citeseer, 2009.

[11] Marc Graniou, Hakan Sivencrona, and Rickard Svenningsson. Advantages and challenges of introducing autosar for safety-related systems. Technical report, SAE Technical Paper, 2009.

[12] Mohamed Ali Shajahan, Nicholas Richardson, Niravkumar Dhameliya, Bhavik Patel, Sunil Kumar Reddy Anumandla, and Vamsi Krishna Yarlagadda. Autosar classic vs. autosar adaptive: A comparative analysis in stack development. *Engineering International*, 7(2):161–178, 2019.

[13] N Rama Venkateswaran, MS Vishwanatha, and Prasannkumar Y Shivayogi. Safety mechanism implementation for fault in the memory. In *2021 IEEE Mysore Sub Section International Conference (MysuruCon)*, pages 210–214. IEEE, 2021.

[14] Infineon Technologies. *AURIX TC3xx User Manual: Part 1*, 2020. Accessed: 2024-09-04.